Деревья решений (Decision Trees) в scikit-learn

1. Общая идея метода

Дерево решений представляет собой модель машинного обучения, которая последовательно делит пространство признаков на области, внутри которых принимается решение о принадлежности объекта к классу или вычисляется численное значение целевой переменной.
В основе метода лежит принцип последовательного выбора наиболее удачного разбиения данных. На каждом шаге алгоритм анализирует признаки и подбирает такое условие, которое лучше всего разделяет объекты.

Если рассматривать задачу классификации, то в конечных листьях дерева размещаются объекты, максимально однородные по классам. Если речь идет о регрессии, то в листьях оказываются объекты с близкими значениями целевой переменной.

Преимущество деревьев решений состоит в том, что логика модели хорошо интерпретируется человеком. Можно проследить путь от корня дерева до листа и увидеть, по каким именно правилам был получен результат.

2. Математическая основа

Пусть в узле m находится множество объектов Qm, а число объектов в этом узле равно nm.
Каждое разбиение определяется парой параметров:

- j - индекс признака
- tm - порог разбиения

Тогда множество объектов делится на две части:

- Qleft(θ) - объекты, для которых xj≤t=m
- Qright(θ) - объекты, для которых xj>tm

Здесь θ=(j,tm).

Качество разбиения оценивается через функцию:

$$G(Q_m, \theta) = \frac{n_{left}}{n_m} H(Q_{left}) + \frac{n_{right}}{n_m} H(Q_{right})$$

где:

- H(Q) - мера неоднородности узла

- nleft и nright- число объектов в левом и правом поддеревьях

Алгоритм ищет такое разбиение, при котором значение G(Qm,θ) становится минимальным.
Именно это означает, что после разделения узлы становятся более "чистыми".

3. Критерии качества разбиения

3.1. Для классификации

Для классификационных задач в узле оценивается доля объектов каждого класса:

$$p_{mk} = \frac{1}{n_m} \sum I(y = k)$$

где pmk - доля объектов класса k в узле m.

На практике чаще всего применяются два критерия.

Индекс Джини:

$$H(Q_m) = \sum_k p_{mk}(1 - p_{mk})$$

Чем меньше индекс Джини, тем более однороден узел.

Энтропия:

$$H(Q_m) = -\sum_k p_{mk} \log(p_{mk})$$

Энтропия также показывает степень неопределенности.
Если в узле находятся объекты одного класса, значение критерия минимально.

3.2. Для регрессии

В регрессионных задачах дерево стремится разбивать данные так, чтобы значения целевой переменной внутри листа были как можно ближе друг к другу.

Среднеквадратичная ошибка:

$$\bar{y}_m = \frac{1}{n_m} \sum y_i$$

$$H(Q_m) = \frac{1}{n_m} \sum (y_i - \bar{y}_m)^2$$

Средняя абсолютная ошибка:

$$H(Q_m) = \frac{1}{n_m} \sum |y_i - \text{median}(y)_m|$$

Таким образом, в классификации дерево уменьшает смешение классов, а в регрессии - разброс численных значений.

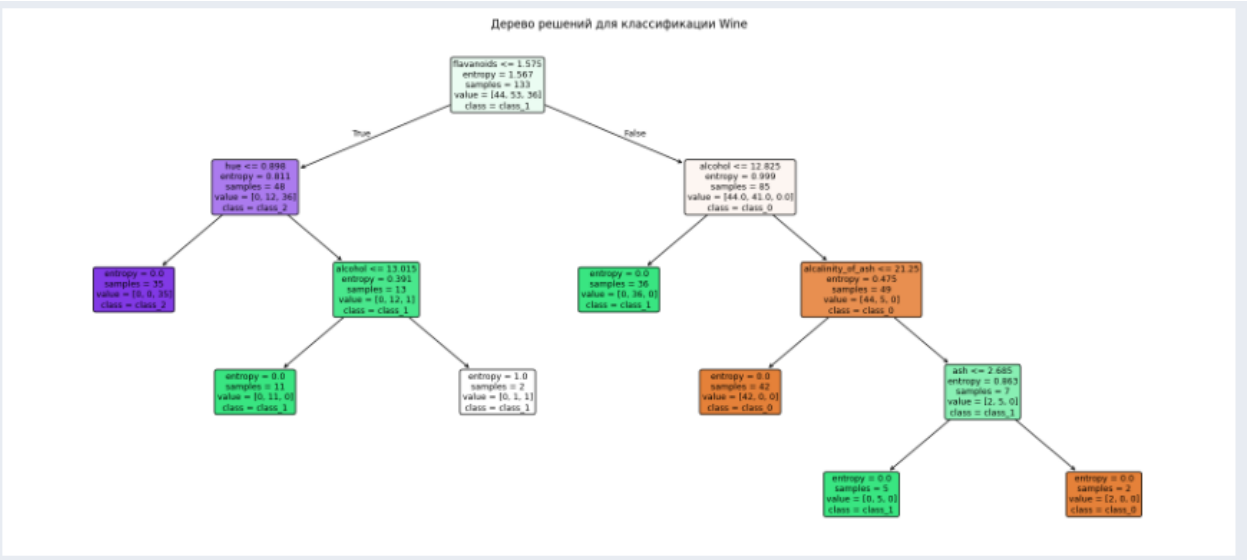
4. Практическая реализация в scikit-learn

Задача 1. Классификация на наборе данных Wine

В качестве первого примера рассмотрим задачу классификации.
Используем набор Wine из библиотеки scikit-learn, обучим дерево решений и посмотрим, какие признаки модель использует в первую очередь.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

# Загрузка данных
wine = load_wine()
X, y = wine.data, wine.target

# Разделение выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

```
# Создание модели
clf = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42
)

# Обучение
clf.fit(X_train, y_train)

# Предсказание
y_pred = clf.predict(X_test)

# Оценка качества
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification report:")
print(classification_report(y_test, y_pred, target_names=wine.target_names))

# Текстовое представление дерева
print("\nТекстовая структура дерева:")
print(export_text(clf, feature_names=list(wine.feature_names)))

# Визуализация дерева
plt.figure(figsize=(18, 8))
plot_tree(
    clf,
    feature_names=wine.feature_names,
    class_names=wine.target_names,
    filled=True,
    rounded=True,
    fontsize=9
)
plt.title("Дерево решений для классификации Wine")
plt.show()
```




```
Точность: 0,8888888888888888

Матрица путаницы:
[[15  0  0]
 [ 3 14  1]
 [ 0  1 11]]

Отчет о классификации:
 точность, полнота, F-мера, поддержка

 class_0 0,83 1,00 0,91 15
 class_1 0,93 0,78 0,85 18
 class_2 0,92 0,92 0,92 12

 точность 0,89 45
 макро среднее 0,89 0,90 0,89 45
средневзвешенное значение 0,90 0,89 0,89 45


Текстовая структура дерева:
|--- флаваноиды <= 1,58
| |--- оттенок <= 0,90
| | |--- класс: 2
| |--- оттенок > 0.90
| | |--- содержание алкоголя <= 13,01
| | | |--- класс: 1
| | |--- алкоголь > 13,01
| | | |--- класс: 1
|--- флаваноиды > 1,58
| |--- содержание алкоголя <= 12,83
| | |--- класс: 1
| |--- алкоголь > 12,83
| | |--- щелочность золы <= 21,25
| | | |--- класс: 0
| | |--- щелочность_of_ash > 21.25
| | | |--- зольность <= 2,69
| | | | |--- класс: 1
| | | |--- зола > 2,69
| | | | |--- класс: 0
```

Пояснение к примеру

В данном случае дерево решает задачу определения сорта вина по химическим характеристикам.

Модель самостоятельно подбирает признаки и пороги, которые позволяют лучше разделить классы.

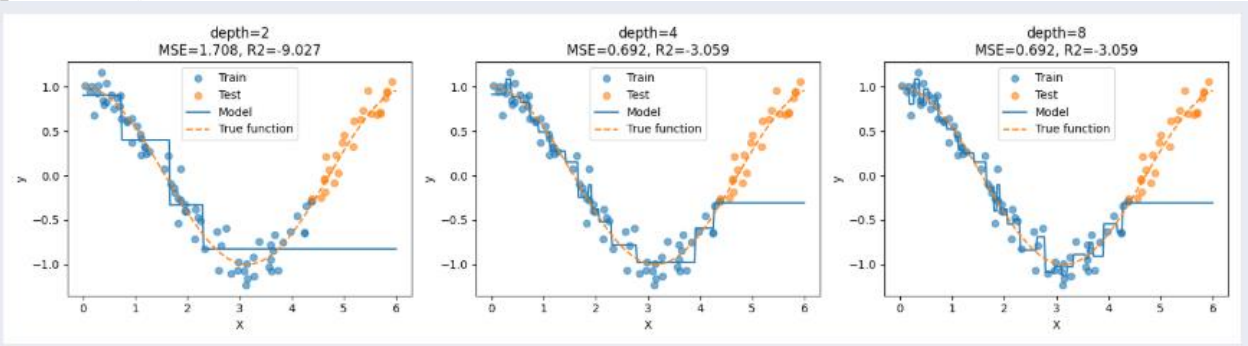
Если после обучения точность оказывается высокой, это означает, что дерево сумело выделить достаточно четкие правила классификации.
При этом слишком большая глубина дерева может привести к переобучению, когда модель слишком точно подстраивается под обучающие данные и начинает хуже работать на тестовой выборке.

Задача 2. Регрессия на искусственно сгенерированных данных

Теперь рассмотрим дерево решений в задаче регрессии.
Построим зависимость с шумом и посмотрим, как меняется поведение модели при разной глубине дерева.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Генерация данных
np.random.seed(42)
X = np.sort(6 * np.random.rand(100, 1), axis=0)
y = np.cos(X).ravel() + np.random.normal(0, 0.15, size=100)

# Разделение на обучающую и тестовую части
X_train, X_test = X[:75], X[75:]
y_train, y_test = y[:75], y[75:]

depths = [2, 4, 8]

plt.figure(figsize=(15, 4))

for i, depth in enumerate(depths, 1):
    reg = DecisionTreeRegressor(
        max_depth=depth,
        min_samples_split=4,
        min_samples_leaf=2,
        random_state=42
    )

    reg.fit(X_train, y_train)
    y_pred = reg.predict(X_test)
```

```
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    X_line = np.linspace(0, 6, 300).reshape(-1, 1)
    y_line = reg.predict(X_line)

    plt.subplot(1, 3, i)
    plt.scatter(X_train, y_train, alpha=0.6, label="Train")
    plt.scatter(X_test, y_test, alpha=0.6, label="Test")
    plt.plot(X_line, y_line, label="Model")
    plt.plot(X_line, np.cos(X_line), linestyle="--", label="True function")
    plt.title(f"depth={depth}\nMSE={mse:.3f}, R2={r2:.3f}")
    plt.xlabel("X")
    plt.ylabel("y")
    plt.legend()

plt.tight_layout()
plt.show()
```



Пояснение к примеру

В регрессии дерево решений строит кусочно-постоянную функцию.
Это означает, что предсказание модели внутри отдельного листа одинаково для всех объектов, попавших в данный лист.

Если глубина дерева небольшая, модель описывает зависимость слишком грубо.
Если глубина слишком большая, дерево начинает повторять шум обучающей выборки.
Следовательно, необходимо подбирать глубину и другие параметры так, чтобы получить разумный компромисс между простотой и точностью.

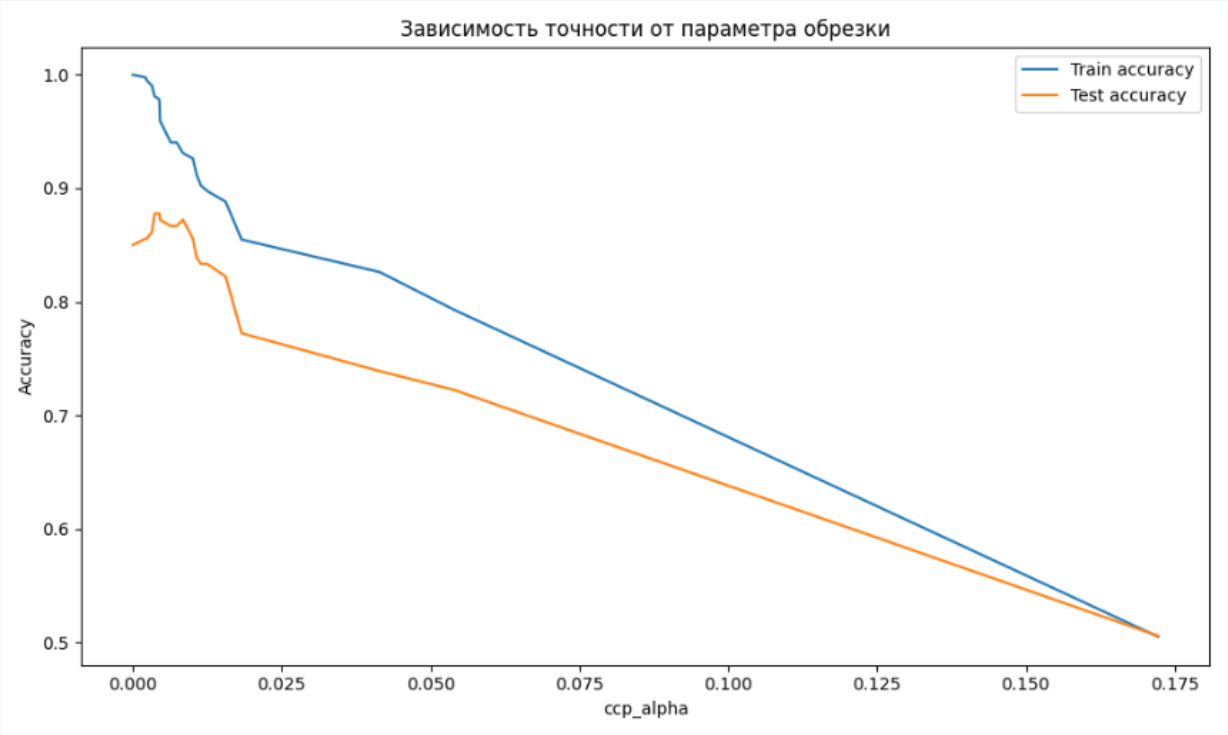
Задача 3. Оценка влияния обрезки дерева

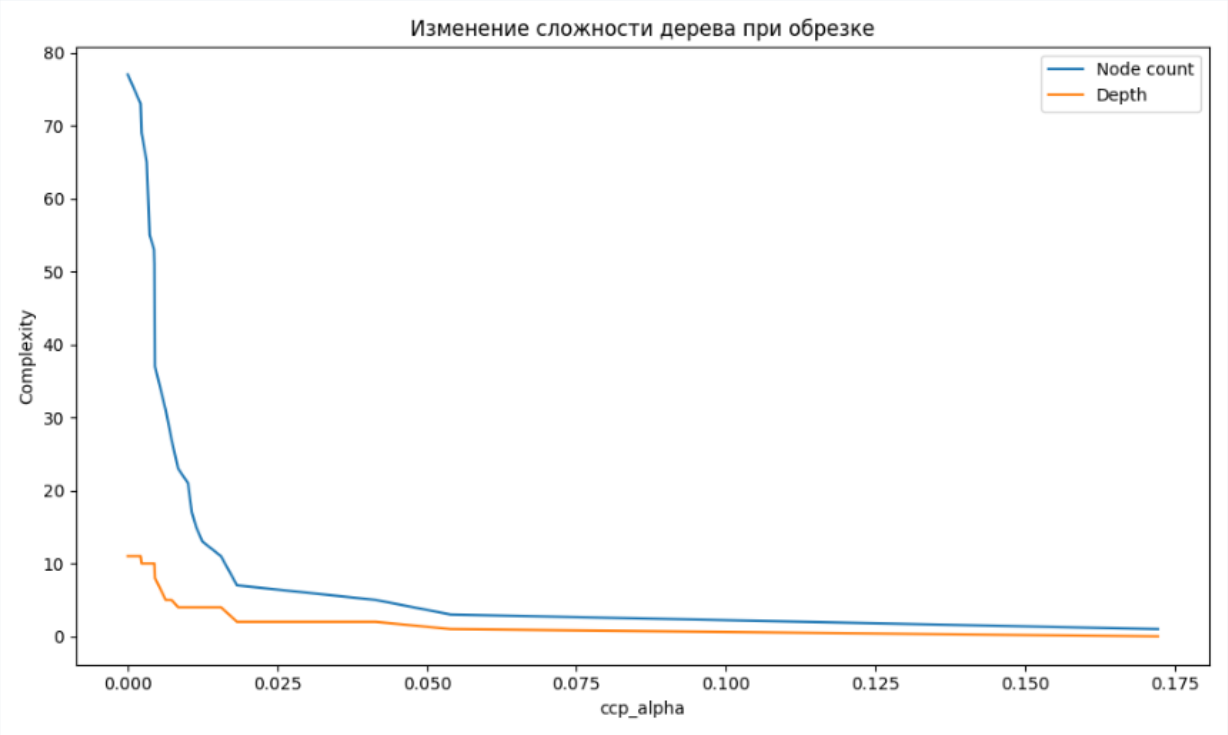
Для деревьев решений особенно важен контроль сложности модели.
Один из способов - cost-complexity pruning, то есть обрезка дерева через параметр ccp_alpha.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Генерация данных
X, y = make_classification(
    n_samples=600,
    n_features=12,
    n_informative=6,
    n_redundant=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

# Базовое дерево
base_clf = DecisionTreeClassifier(random_state=42)
base_clf.fit(X_train, y_train)

print("Без обрезки:")
print("Количество узлов:", base_clf.tree_.node_count)
print("Глубина дерева:", base_clf.tree_.max_depth)

# Путь обрезки
path = base_clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

models = []
train_scores = []
test_scores = []
node_counts = []
depths = []

for alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
    clf.fit(X_train, y_train)

    models.append(clf)
```

```
    train_scores.append(clf.score(X_train, y_train))
    test_scores.append(clf.score(X_test, y_test))
    node_counts.append(clf.tree_.node_count)
    depths.append(clf.tree_.max_depth)

best_index = np.argmax(test_scores)
best_alpha = ccp_alphas[best_index]

print("\nЛучший alpha:", best_alpha)
print("Точность на тесте:", test_scores[best_index])

plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas, train_scores, label="Train accuracy")
plt.plot(ccp_alphas, test_scores, label="Test accuracy")
plt.xlabel("ccp_alpha")
plt.ylabel("Accuracy")
plt.title("Зависимость точности от параметра обрезки")
plt.legend()
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas, node_counts, label="Node count")
plt.plot(ccp_alphas, depths, label="Depth")
plt.xlabel("ccp_alpha")
plt.ylabel("Complexity")
plt.title("Изменение сложности дерева при обрезке")
plt.legend()
plt.show()
```

```
Без обрезки:
Количество узлов: 77
Глубина дерева: 11

Лучший альфа-показатель: 0,00366374286941054
Точность теста: 0,8777777777777778
```





Пояснение к примеру

При увеличении ccp_alpha дерево становится проще: уменьшается число узлов и сокращается глубина.

С одной стороны, это снижает вероятность переобучения.
С другой стороны, чрезмерная обрезка может привести к тому, что дерево станет слишком грубым и потеряет важные закономерности.

Поэтому в реальной задаче параметр ccp_alpha следует подбирать по качеству на валидационной или тестовой выборке.

5. Основные параметры дерева решений

Ниже приведены наиболее важные параметры, которые чаще всего используются при настройке модели.

max_depth

Ограничивает максимальную глубину дерева.
Чем больше глубина, тем сложнее модель.
Если глубина не ограничена, дерево может слишком точно подстроиться под обучающие данные.

min_samples_split

Задает минимальное число объектов, необходимое для разбиения узла.
Если объектов меньше этого значения, дальнейшее разбиение не выполняется.

min_samples_leaf

Определяет минимальное число объектов в листе.
Параметр помогает избежать слишком маленьких листьев, которые делают модель нестабильной.

criterion

Указывает критерий качества разбиения.

Для классификации обычно используются:

- gini
- entropy

Для регрессии применяются, например:

- squared_error
- absolute_error

ccp_alpha

Параметр обрезки дерева.
Позволяет уменьшить сложность модели уже после построения полной структуры дерева.

6. Интерпретация результатов

Одно из важных достоинств дерева решений связано с возможностью интерпретации.

6.1. Важность признаков

Модель позволяет определить, какие признаки сильнее всего влияли на итоговые разбиения.

```
import matplotlib.pyplot as plt

feature_importance = clf.feature_importances_

for name, value in zip(wine.feature_names, feature_importance):
    print(f"{name}: {value:.4f}")

plt.figure(figsize=(10, 4))
plt.bar(wine.feature_names, feature_importance)
plt.xticks(rotation=45)
plt.title("Важность признаков")
plt.tight_layout()
plt.show()
```

Если некоторый признак имеет высокую важность, это означает, что дерево часто использовало его в верхней части структуры и он заметно влиял на качество разбиений.

6.2. Анализ структуры дерева

Также можно получить техническую информацию о внутреннем устройстве дерева.

```
n_nodes = clf.tree_.node_count
children_left = clf.tree_.children_left
children_right = clf.tree_.children_right
feature = clf.tree_.feature
threshold = clf.tree_.threshold

print("Количество узлов:", n_nodes)

for i in range(n_nodes):
```

```
    if children_left[i] != children_right[i]:
        print(f"Узел {i}: признак {feature[i]} <= {threshold[i]:.3f}")
```

Такой анализ полезен, когда нужно понять, как именно модель строит правила классификации или регрессии.

7. Задачи для проверки понимания

Задание 1

Почему дерево решений относят к интерпретируемым моделям?

Ответ:
Потому что его решение можно представить как последовательность понятных условий вида "если признак меньше или равен порогу, перейти влево, иначе вправо". Пользователь может проследить путь от корня до листа и увидеть, по каким правилам модель пришла к результату.

Задание 2

Для чего нужен параметр max_depth?

Ответ:
Он ограничивает глубину дерева и тем самым контролирует сложность модели. При слишком большой глубине возрастает риск переобучения, а при слишком маленькой дерево может оказаться слишком простым.

Задание 3

В чем различие применения дерева решений в классификации и регрессии?

Ответ:
В классификации дерево стремится разделить объекты по классам так, чтобы узлы были максимально однородными по меткам. В регрессии дерево делит объекты так, чтобы значения целевой переменной внутри листьев были как можно ближе друг к другу.

Задание 4

Что показывает важность признаков?

Ответ:
Она показывает относительный вклад признаков в построение дерева. Чем выше значение важности, тем сильнее данный признак влиял на разбиения и итоговые решения модели.

Задание 5

Зачем нужна обрезка дерева через ccp_alpha?

Ответ:
Она используется для уменьшения сложности дерева после его построения. Обрезка помогает сократить количество узлов, уменьшить глубину и снизить риск переобучения.

8. Выводы

1. Деревья решений являются наглядным и удобным методом машинного обучения, который применим как к классификации, так и к регрессии.
2. Основа метода состоит в последовательном выборе таких разбиений, которые уменьшают неоднородность данных в узлах.
3. Для классификации обычно используются критерии Джини и энтропии, а для регрессии - меры разброса, например среднеквадратичная ошибка.
4. Практическое качество дерева сильно зависит от параметров глубины, минимального числа объектов в узлах и параметра обрезки.
5. Существенным преимуществом метода является интерпретируемость, так как пользователь может анализировать структуру дерева, важность признаков и правила, по которым модель принимает решение.